SYSTEM GUIDE · TASK_KEEPER

Architecture & Dependencies

How the application is structured, how its runtime pieces work together, and why each direct dependency is present.

A source-based guide to the browser client, API, scheduler, persistence model, Redis caching, integrations, and local/production runtime.

Updated November 1, 2026 · Reflects the current source, workspace manifests, and Redis read-cache implementation

01 · SYSTEM ARCHITECTURE

One web app, two runtime workspaces

The root npm workspace coordinates a React client and an Express server. The server owns task rules, persistence, authentication, and a background reminder scheduler. MongoDB is required; Redis and outbound delivery channels are optional.

Browser · React 19

`client/src/App.jsx`
Account and task views, task actions, history, notifications, theme, foreground alerts.

API adapter · same-origin HTTP

`client/src/api.js`
JSON requests to `/api`, cookies and friendly network/server errors.

Express API · Node.js

`server/src/app.js`
Authentication, task/occurrence and notification routes, middleware and error handling.

MongoDB · required

Users, tasks, occurrences, notifications, push subscriptions, delivery outbox, scheduler lease and indexes.

Scheduler · same API process

Starts after MongoDB connects; ticks every 15 seconds and uses a MongoDB lease across API instances.

Redis + delivery · optional

Redis shares auth limits and caches selected reads; SMTP email and VAPID Web Push are optional channels.

Mode	How requests and UI are served	Runtime behavior
Development	Vite serves the client at `localhost:5173` and proxies `/api` to Express at `localhost:3001`.	Root `concurrently` starts Vite and the Node watch process.
Production	Vite builds the client into repository-root `build/`; Express serves it with the API when the folder exists.	One Node API process runs the scheduler. MongoDB is an external required service.

> **Request flow:** browser → same-origin JSON request + HTTP-only session cookie → Express authentication and route validation → user-scoped MongoDB read/write → API response. Redis can serve a recent cached read without querying MongoDB.

Security boundaries: authenticated data access is scoped by user ID. Session JWTs are signed with `JWT_SECRET`, stored in a seven-day HTTP-only, SameSite=Lax cookie, and marked Secure in production. Common security headers and a JSON body-size limit are set by Express.

02 · DATA AND BACKGROUND WORK

Schedules become durable occurrences

A task is a recurrence definition; an occurrence is one scheduled instance. This separation preserves history and gives each instance its own completion, missed, snooze, and reminder lifecycle.

MongoDB collection	Stores / purpose
`users`	Identity, bcrypt password hash and embedded notification preferences.
`tasks`	Owner, schedule/frequency settings, timezone, next due instant and archive state.
`occurrences`	Scheduled instance, task-title/timezone snapshot, lifecycle timestamps, snooze and reminder state.
`notifications`	Durable in-app messages, owner, read state and optional occurrence link.
`pushSubscriptions`	Browser push endpoint and keys, associated with a user.
`notificationDeliveries`	Retryable per-channel email/push outbox state, attempts and retry time.
`schedulerLocks`	Lease/heartbeat that coordinates scheduler ticks across API instances.

Scheduler lifecycle

1. After MongoDB connects, the API starts scheduler ticks every 15 seconds.
2. It fills a rolling seven-day occurrence horizon and keeps a future open occurrence for active tasks.
3. It marks overdue occurrences missed, creates reminders, handles snooze expiry, and repeats missed reminders every 10 minutes while open.
4. It claims eligible email/push outbox entries, sends them, and records success or retry state.
5. Task/occurrence/notification writes invalidate the affected user's Redis read-cache version.

Time and recurrence

Daily, weekly, monthly and yearly schedules use local time and an IANA timezone. Server-side `Intl.DateTimeFormat` preserves local due time across daylight-saving changes. Monthly dates clamp to month-end; yearly February 29 is rejected.

Integrity and indexes

Mongo documents use UUID string IDs and camelCase fields; API responses map to snake_case. Startup indexes enforce unique email, occurrence and delivery keys and accelerate queue/history queries. Multi-document transactions require a MongoDB replica set.

> **Availability:** the API opens its HTTP listener first, then connects/retries MongoDB every five seconds. The scheduler starts after MongoDB becomes available. Account and task data still require MongoDB.

03 · CLIENT DEPENDENCIES

Browser workspace

Direct dependencies and version ranges declared in `client/package.json`.

Package	Range	Use	Why it is needed
react	`^19.1.0`	`App.jsx` components and hooks.	Builds interactive account, task, history and notification UI with local component state.
react-dom	`^19.1.0`	`main.jsx` via `createRoot`.	Mounts the React component tree into the browser DOM.
vite	`^6.3.5`	Client scripts and `vite.config.js`.	Development server, API proxy, and production client bundler.
@vitejs/plugin-react	`^4.7.0`	Registered in `vite.config.js`.	Provides React/JSX support in Vite.

Platform APIs (not npm packages)

Capability	Role
Fetch, HTTP cookies, JSON	Same-origin browser/API transport and session-cookie handling.
`Intl.DateTimeFormat`	Detects browser timezone and formats due/history dates.
localStorage + DOM	Persists theme choice and updates theme/page title.
Notifications + Service Worker	Foreground alerts and background push event/click handling.
Push API	Creates browser subscriptions for optional VAPID push.
CSS	Responsive layout, themes and visual design; no CSS framework is declared.

`client/src/api.js` handles same-origin JSON requests, credentials, 204 responses and friendly error mapping. `client/public/sw.js` handles push events and notification clicks. In-app records and foreground alerts do not require VAPID; background push does.

04 · SERVER DEPENDENCIES

API workspace

Direct dependencies and version ranges declared in `server/package.json`.

Package	Range	Use and reason
express	`^5.1.0`	HTTP application, middleware, routers and API endpoints.
mongodb	`^7.7.0`	Official database driver for connection pool, queries, indexes, sessions and transactions. MongoDB is required.
bcryptjs	`^3.0.2`	Hashes registration passwords and verifies login passwords; avoids storing plaintext passwords.
jsonwebtoken	`^9.0.2`	Signs and verifies the session JWT stored in the HTTP-only cookie.
cookie-parser	`^1.4.7`	Parses request cookies so authentication middleware can read the session token.
express-rate-limit	`^7.5.0`	Limits registration/login attempts; supports Redis or process-local store.
redis	`^6.3.1`	Optional Redis client for shared auth limits and user-scoped read caching.
rate-limit-redis	`^4.3.1`	Adapts Redis commands to the rate-limit store interface.
zod	`^4.6.7`	Validates request data, task recurrence constraints, preferences and subscriptions.
dotenv	`^16.5.0`	Loads private local environment configuration or the configured `ENV_FILE`.
nodemailer	`^10.2.1`	Sends optional queued email notifications over SMTP.
web-push	`^3.6.7`	Sends optional background browser push messages signed with VAPID.

Root development dependency

Package	Range	Why it is needed
concurrently	`^9.2.1`	Starts the API watch process and Vite development server together from root `npm run dev`.

> **Node built-ins:** `node:http` creates the listener; `node:crypto` generates UUIDs; filesystem/path/URL modules resolve static assets and environment-file locations. These are part of Node.js, not npm dependencies.

Where Redis helps—and what it does not do

Redis is optional. When configured, it serves two separate purposes: shared authentication rate-limit counters and short-lived response caching to reduce repeated MongoDB reads.

Cached GET response	Cache key scope	Invalidated after
Active and archived task lists	Authenticated user + active/archive selector	Task create/edit/archive/restore/delete, occurrence complete/snooze, scheduler changes.
Paginated task history	Authenticated user + page size/offset	Task/occurrence lifecycle changes and history deletes/clears.
Recent notifications	Authenticated user	Reminder/schedule notification creation, read/delete/clear and related task/history changes.
Notification settings	Authenticated user	Preference updates.

- Entries expire after 10 seconds as a safety bound. Per-user version keys invalidate all of that user's cached responses after relevant writes, including scheduler-created changes.
- Authentication/session endpoints and writes are not cached. Cache keys include the authenticated user ID to keep accounts isolated.
- Without `REDIS_URL`, or when a cache read fails, reads go directly to MongoDB. Auth rate limits fall back to process-local memory.
- If cache invalidation fails, the entry's 10-second expiry bounds staleness. Cache errors are logged and do not block normal database-backed reads/writes.
- This reduces repeated database work; it does not guarantee lower latency in every deployment. Redis must be configured and reachable, and gains depend on read repetition and service latency.

Required and optional runtime settings

Setting	Purpose
`MONGODB_URI`	Required MongoDB connection string; keep credentials private.
`JWT_SECRET`	Required session signing secret of at least 32 characters.
`REDIS_URL`	Optional Redis connection for shared auth limits and read cache; unset by default.
`MONGODB_DATABASE`, `DB_POOL_MAX`, `PORT`, `ENV_FILE`	Optional database name, pool cap, listen port (3001 default), alternate environment file.
`EMAIL_SMTP_*`, `EMAIL_FROM`	Optional SMTP delivery configuration.
`VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`	Optional Web Push application-server configuration; keep private key secret.

> **Deployment:** Node.js 20.19+ and a MongoDB replica set are required. Redis is not required for normal task usage. The API retries MongoDB connection at startup and must remain running for scheduler reminders. Build the client before production and use HTTPS.

Source references: `package.json`, `client/package.json`, `server/package.json`, `server/src`, `client/src`, `README.md`, and `overview.md`. Version ranges are the declarations in the workspace manifests; browser and Node built-ins are not npm packages.